The comparison between the values in Tables 16.01 and 16.03 illustrate the greater range of positive and negative values available if floating-point representation is used.

Extension Question 16.02

a Using the methods suggested in Chapter 1 (Section 1.01) can you confirm for yourself that the denary equivalents of the binary codes shown in Tables 16.02 and Table 16.03 are as indicated?

b Can you also confirm that conversion from positive to negative (or the conversion from negative to positive) for a fixed-format real value still follows the rules defined in Chapter 1 (Section 1.02) for two's complement representation?

Precision and normalisation

You have to decide about the format of a floating-point representation in two respects. You have to decide the total number of bits to be used and decide on the split between those representing the mantissa and those representing the exponent. In practice, a choice for the total number of bits to be used will be available as an option when the program is written. However, the split between the two parts of the representation will have been determined by the floating-point processor. If you did have a choice you would base your decision on the fact that increasing the number of bits for the mantissa would give better precision for a value stored but would leave fewer bits for the exponent, which reduces the range of possible values.

To achieve maximum precision you have to normalise a floating-point number. (This normalisation is unrelated to the process associated with designing a database.) Precision increases with an increasing number of bits for the mantissa, so optimum precision will only be achieved if full use is made of these bits. In practice, that means using the largest possible magnitude for the value represented by the mantissa.

To illustrate this, we can consider the eight-bit representation used in Table 16.03. Table 16.04 shows possible representations for denary 2 using this representation.

Denary representation	Floating-point binary representation
0.125×2^4	0 001 0100
0.25×2^3	0 010 0011
0.5×2^2	0 100 0010

Table 16.04 Alternative representations of denary 2 using four bits each for mantissa and exponent

For a negative number we can consider representations for –4 as shown in Table 16.05.

Denary representation	Floating-point binary representation
-0.25×2^4	1 110 0100
-0.5×2^3	1 100 0011
-1.0×2^2	1 000 0010

Table 16.05 Alternative representations of denary –4 using four bits each for mantissa and exponent

When the number is represented with the highest magnitude for the mantissa, the two most significant bits are different. This fact can be used to recognise that a number is in a normalised representation. The values in Tables 16.03 and 16.04 also show how a number

could be normalised. For a positive number, the bits in the mantissa are shifted left until the most significant bits are 0 followed by 1. For each shift left the value of the exponent is reduced by 1.

The same process of shifting is used for a negative number until the most significant bits are 1 followed by 0. In this case, no attention is paid to the fact that bits are falling off the most significant end of the mantissa.

Extension Question 16.03

Look at the data in Table 16.03. Do you see any conflict with the above discussion? What is likely to be the approach in a typical floating-point system?

Conversion of representations

In Chapter 1 (Section 1.01), a number of methods for converting numbers into different representations were discussed. These only considered integer values. We now need to consider the conversion of real numbers.

We can start by considering the conversion of a simple real number, such as 4.75, into a simple fixed-point binary representation. This looks easy because 4 converts to 100 in binary and .75 converts to .11 in binary so the binary version of 4.75 should be:

100.11

However, remember that a positive number should start with 0. Can we just add a sign bit? For a positive number we can. Denary 4.75 can be represented as 0100.11 in binary.

For negative numbers we still want to use two's complement form. So, to find the representation of –4.75 we can start with the representation for 4.75 then convert it to two's complement as follows:

0100.11 converts to 1011.00 in one's complement

then to 1011.01 in two's complement

To check the result, we can apply Method 2 from Worked Example 1.01 in Chapter 1. 1011 is the code for –8 + 3 and .01 is the code for .25; –8 + 3 + .25 = –4.75.

We can now consider the conversion of a denary value expressed as a real number into a floating-point binary representation. Before considering the conversion method it should be remembered that most fractional parts do not convert to a precise representation. This is because the binary fractional parts represent a half, a quarter, an eighth, a sixteenth and so on. Unless a denary fraction is a sum of a collection of these values, there cannot be an accurate conversion. In particular, of the values from .1 through to .9, only .5 converts accurately. This was mentioned in Chapter 1 (Section 1.03) in the discussion about storing currency values.

The method for conversion of a positive value is as follows.

1. Convert the whole-number part using the method described in Chapter 1 (Section 1.01).
2. Add the 0 sign bit.
3. Convert the fractional part choosing a method from one of the examples in Worked Example 16.01.
4. Combine the whole number and fractional parts and enter these into the most significant of the bits allocated for the representation of the mantissa.

5 Fill the remaining bits for the mantissa and the bits for the exponent with zeros.

6 Adjust the position of the binary point by changing the exponent value to achieve a normalised representation.

To convert a negative value the number is treated initially as positive and the same first five steps are followed. At this stage a two's complement conversion of the mantissa code is used to convert this to a negative value before step 6 is carried out.

WORKED EXAMPLE 16.01

Converting a denary value to a floating-point representation

Example 1

Let's consider the conversion of 8.75.

1 The 8 converts to 1000, adding the sign bit gives 01000.

2 The .75 can be recognised as being .11 in binary.

3 The combination gives 01000.11 which has exponent value zero.

4 Shifting the binary point gives 0.100011 which has exponent value denary 4.

5 The next stage depends on the number of bits defined for the mantissa and the exponent; if ten bits are allocated for the mantissa and four bits are allocated for the exponent the final representation becomes 0100011000 for the mantissa and 0100 for the exponent.

Example 2

Let's consider the conversion of 8.63. The first step is the same but now the .63 has to be converted by the 'multiply by two and record whole number parts' method. This works as follows:

.63 × 2 = 1.26 so 1 is stored to give the fraction .1

.26 × 2 = .52 so 0 is stored to give the fraction .10

.52 × 2 = 1.04 so 1 is stored to give the fraction .101

.04 × 2 = .08 so 0 is stored to give the fraction .1010

At this stage it can be seen that, multiplying .08 by 2 successively is going to give a lot of zeros in the binary fraction before another 1 is added so the process can be stopped. .63 has been approximated as .625. So, following Steps 3–5 in Example 1, the final representation becomes 0100010100 for the mantissa and 0100 for the exponent.

TASK 16.01

Convert the denary value –7.75 to a floating-point binary representation with ten bits for the mantissa and four bits for the exponent. Start by converting 7.75 to binary (make sure you add the sign bit!). Then convert to two's complement form. Finally, choose the correct value for the exponent to leave the implied position of the binary point after the sign bit. Convert back to denary to check the result.

Problems with using floating-point numbers

As illustrated above, the conversion of a real value in denary to a binary representation almost guarantees a degree of approximation. There is also a restriction of the number of bits used to store the mantissa.

Floating-point numbers are used in extended mathematical procedures involving repeated calculations. For example, in weather forecasting using a mathematical model of the atmosphere, or in economic forecasting. In such programming there is a slight approximation in recording the result of each calculation. These so-called rounding errors can become significant if calculations are repeated enough times. The only way of preventing the errors becoming a serious problem is to increase the precision of the floating-point representation by using more bits for the mantissa. Programming languages therefore offer options to work in 'double precision' or 'quadruple precision'.

The other potential problem relates to the range of numbers that can be stored. Referring back to the simple eight-bit representation illustrated in Table 16.03, the highest value represented is denary 112. A calculation can easily produce a value higher than this. As Chapter 1 (Section 1.02) illustrated, this produces an overflow error condition. However, for floating-point values there is also a possibility that if a very small number is divided by a number greater than 1 the result is a value smaller than the smallest that can be stored. This is an underflow error condition. Depending on the circumstances, it may be possible for a program to continue running by converting this very small number to zero but this must involve risk.

Summary

- Examples of non-composite user-defined data types include enumerated and pointer data types.
- Record, set and class are examples of composite user-defined data types.
- File organisation allows for serial, sequential or direct access.
- Floating-point representation for a real number allows a wider range of values to be represented.
- A normalised floating-point representation achieves the best precision for the value stored.
- Stored floating-point values rarely give an accurate representation of the denary equivalent.

Reflection Point:

Whenever you are asked to create a binary representation from a denary value or vice-versa are you always checking your answer by converting it back to the original value?